

Solution 1

Solution 1 (a)

Given the probability distribution P over binary variables $x_1, \dots, x_6 \in \{0, 1\}$ has the following form:

$$P(x_1, \dots, x_6) = \frac{1}{Z} \psi(x_1, x_2) \psi(x_2, x_3) \psi(x_3, x_4) \psi(x_4, x_5) \psi(x_5, x_6) \psi(x_6, x_1)$$

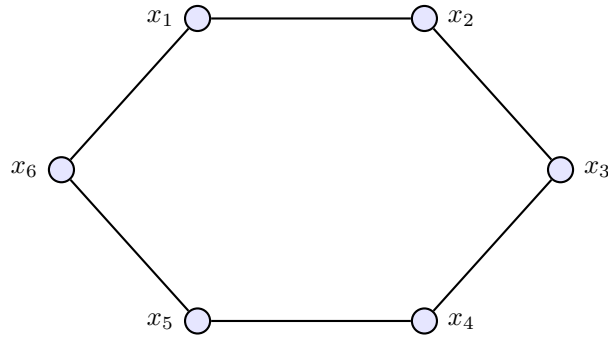
where for $a, b \in \{0, 1\}$,

$$\psi(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0.5 & \text{otherwise} \end{cases}$$

From the Hammersley-Clifford theorem the potential terms are:

$$\begin{aligned} \psi(x_1, x_2) &\implies \text{edge between } x_1 \text{ and } x_2 & \psi(x_4, x_5) &\implies \text{edge between } x_4 \text{ and } x_5 \\ \psi(x_2, x_3) &\implies \text{edge between } x_2 \text{ and } x_3 & \psi(x_5, x_6) &\implies \text{edge between } x_5 \text{ and } x_6 \\ \psi(x_3, x_4) &\implies \text{edge between } x_3 \text{ and } x_4 & \psi(x_6, x_1) &\implies \text{edge between } x_6 \text{ and } x_1 \end{aligned}$$

Now, with the information above, we can draw the minimal undirected graph.



$\therefore$ The minimal undirected graph given the probability distribution P is seen above.

Solution 1

Solution 1 (b)

Given the probability distribution P over binary variables $x_1, \dots, x_6 \in \{0, 1\}$ has the following form:

$$P(x_1, \dots, x_6) = \frac{1}{Z} \psi(x_1, x_2) \psi(x_2, x_3) \psi(x_3, x_4) \psi(x_4, x_5) \psi(x_5, x_6) \psi(x_6, x_1)$$

where for $a, b \in \{0, 1\}$, and

$$\psi(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0.5 & \text{otherwise} \end{cases}$$

To find the x that maximizes $P(X)$ we have the following:

$$\arg \max_x P(x) = \arg \max_x \left[\frac{1}{Z} \prod_{i=1}^5 \psi(x_i, x_{i+1}) \cdot \psi(x_6, x_1) \right]$$

Since the partition function Z is a positive constant, we can ignore it and maximizing $P(x)$ becomes:

$$\arg \max_x P(x) = \arg \max_x [\psi(x_1, x_2) \psi(x_2, x_3) \psi(x_3, x_4) \psi(x_4, x_5) \psi(x_5, x_6) \psi(x_6, x_1)]$$

Given $\psi(a, b)$ is maximized at, the maximum possible value for any single potential is $\psi(a, a) = 1$.

$$\psi(a, b) = 1 \text{ where } a = b$$

Now, maximizing the value of each term in the product yields:

$$\psi(x_1, x_2) = 1 \implies x_1 = x_2 \quad \psi(x_4, x_5) = 1 \implies x_4 = x_5$$

$$\psi(x_2, x_3) = 1 \implies x_2 = x_3 \quad \psi(x_5, x_6) = 1 \implies x_5 = x_6$$

$$\psi(x_3, x_4) = 1 \implies x_3 = x_4 \quad \psi(x_6, x_1) = 1 \implies x_6 = x_1$$

Hence, $x_1 = x_2 = x_3 = x_4 = x_5 = x_6$ is the condition for the configuration that maximizes of $P(x)$.

$\therefore$ Since x_i can only be 0 or 1 ($x_i \in \{0, 1\}$), the configurations that maximize $P(x)$ are:

$$x = (0, 0, 0, 0, 0, 0) \quad \text{and} \quad x = (1, 1, 1, 1, 1, 1)$$

Solution 1

Solution 1 (c)

Given the probability distribution P over binary variables $x_1, \dots, x_6 \in \{0, 1\}$ has the following form:

$$P(x_1, \dots, x_6) = \frac{1}{Z} \psi(x_1, x_2) \psi(x_2, x_3) \psi(x_3, x_4) \psi(x_4, x_5) \psi(x_5, x_6) \psi(x_6, x_1)$$

where for $a, b \in \{0, 1\}$, and

$$\psi(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0.5 & \text{otherwise} \end{cases}$$

To find the x that minimizes $P(X)$ we have the following:

$$\arg \min_x P(x) = \arg \min_x \left[\frac{1}{Z} \prod_{i=1}^5 \psi(x_i, x_{i+1}) \cdot \psi(x_6, x_1) \right]$$

Since the partition function Z is a positive constant, we can ignore it and minimizing $P(x)$ becomes:

$$\arg \min_x P(x) = \arg \min_x [\psi(x_1, x_2) \psi(x_2, x_3) \psi(x_3, x_4) \psi(x_4, x_5) \psi(x_5, x_6) \psi(x_6, x_1)]$$

Given $\psi(a, b)$ is minimized at:

$$\psi(a, b) = 0.5 \text{ where } a \neq b$$

Now, minimizing the value of each term in the product yields:

$$\psi(x_1, x_2) = 0.5 \implies x_1 \neq x_2 \implies x_1 = 0, x_2 = 1 \text{ OR } x_1 = 1, x_2 = 0$$

$$\psi(x_2, x_3) = 0.5 \implies x_2 \neq x_3 \implies x_2 = 0, x_3 = 1 \text{ OR } x_2 = 1, x_3 = 0$$

$$\psi(x_3, x_4) = 0.5 \implies x_3 \neq x_4 \implies x_3 = 0, x_4 = 1 \text{ OR } x_3 = 1, x_4 = 0$$

$$\psi(x_4, x_5) = 0.5 \implies x_4 \neq x_5 \implies x_4 = 0, x_5 = 1 \text{ OR } x_4 = 1, x_5 = 0$$

$$\psi(x_5, x_6) = 0.5 \implies x_5 \neq x_6 \implies x_5 = 0, x_6 = 1 \text{ OR } x_5 = 1, x_6 = 0$$

$$\psi(x_6, x_1) = 0.5 \implies x_6 \neq x_1 \implies x_6 = 0, x_1 = 1 \text{ OR } x_6 = 1, x_1 = 0$$

Hence, the conditions for the configuration that minimizes of $P(x)$ are:

$$x = (0, 1, 0, 1, 0, 1) \quad \text{and} \quad x = (1, 0, 1, 0, 1, 0)$$

$\therefore$ Since x_i can only be 0 or 1 ($x_i \in \{0, 1\}$), and $a \neq b$ the configurations that maximize $P(x)$ are:

$$x = (0, 1, 0, 1, 0, 1) \quad \text{and} \quad x = (1, 0, 1, 0, 1, 0)$$

Solution 2

Solution 2 (a)

We will test for independence by finding a counterexample. Let's use $a = 0$ and $c = 1$.

- $A = 0$ is the event "Tom does not win his first game."
- $C = 1$ is the event "Tom wins all three of his games."

The event $C = 1$ (winning all three) requires winning Game 1. The event $A = 0$ requires losing Game 1. These two events are mutually exclusive, so they cannot happen at the same time.

Therefore, the joint probability is $\Pr(A = 0, C = 1) = 0$.

Now, we check the product of the marginal probabilities, $\Pr(A = 0) \Pr(C = 1)$. Assuming the games are non-deterministic (i.e., the probability of winning any game p_i is $0 < p_i < 1$):

- $\Pr(A = 0) = 1 - p_1 > 0$
- $\Pr(C = 1) = p_1 p_2 p_3 > 0$

The product of the marginals is $\Pr(A = 0) \Pr(C = 1) = (1 - p_1)(p_1 p_2 p_3) > 0$.

We have shown: $\Pr(A = 0, C = 1) = 0$ but $\Pr(A = 0) \Pr(C = 1) > 0$. Since $0 \neq \Pr(A = 0) \Pr(C = 1)$, the independence condition is violated.

$\therefore$ No, C is not independent of A .

Solution 2

Solution 2 (b)

Yes, C is conditionally independent of A given B .

The brief reason is that the event B (winning the first two games) already contains all the information from event A (winning the first game).

- If we know $B = 1$ (Tom won G1 and G2), we already know $A = 1$. Thus, knowing A provides no new information.
- If we know $B = 0$ (Tom did not win G1 and G2), we know C must be 0 (he cannot win all three). This is true regardless of A , so again, A provides no new information.

Since $\Pr(C \mid A, B) = \Pr(C \mid B)$ holds in all cases, the statement is true.

$\therefore$ Yes, C is conditionally independent of A given B , because once we know the outcome of the first two games (B), knowing the outcome of *only* the first game (A) provides no new information for predicting the outcome of all three games (C).

Solution 2

Solution 2 (c)

The structure of the minimal undirected graph is determined by the conditional independence relationships between the variables.

- Edge (A, C): From *part (b)*, we know $A \perp C \mid B$. This means that B separates A and C . Hence, there does not exist a direct edge between A and C .
- Edge (A, B): Check if $A \perp B \mid C$. They are not independent, consider if $C = 0$, knowing $A = 0$ implies $B = 0$. Since $A \not\perp B \mid C$, there exists an edge between A and B .
- Edge (B, C): Check if $B \perp C \mid A$. They are not independent, consider if $A = 1$, knowing $B = 0$ (Tom lost G2) implies $C = 0$. Since $B \not\perp C \mid A$, there exists an edge between B and C .

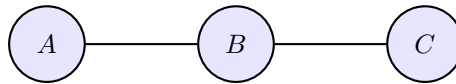


Figure 1: Undirected Graphical Model for (A, B, C)

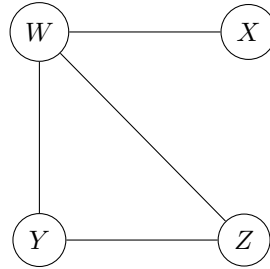
$\therefore$ The undirected graphical model, seen above, encodes the relationship that A and C are conditionally independent given B .

Solution 3

Solution 3 (a)

From the Hammersley-Clifford theorem, the joint probability distribution P that factors over an undirected graph G can be written as a product of potential functions (ψ_C) over the maximal cliques of the graph.

Given, four random variables W, X, Y, Z factor over the graph shown below.



By inspection of the graph above the two maximal cliques are:

- The triangle $\{W, Y, Z\}$
- The edge $\{W, X\}$

Hence, all other cliques,

$$\{W, Y\}, \{W, Z\}, \text{ and } \{Y, Z\}$$

are not maximal, as they are all subsets of the larger clique $\{W, Y, Z\}$.

The joint distribution is the product of potentials over these two maximal cliques, normalized by the partition function Z .

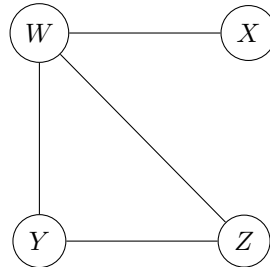
$\therefore$ The functional form is $P(W, X, Y, Z)$, where Z is the partition function, is defined as:

$$P(W, X, Y, Z) = \frac{1}{Z} \psi_1(W, X) \psi_2(W, Y, Z) \text{ such that } Z = \sum_{w, x, y, z} \psi_1(w, x) \psi_2(w, y, z)$$

Solution 3

Solution 3 (b)

Given, four random variables W, X, Y, Z factor over the graph shown below.



Let, $A = \{X\}$ and $B = \{Y, Z\}$

Now, check all paths from A to B to see if $C = \{W\}$ is a separating set.

1. Paths from X to Y are

$$X \rightarrow W \rightarrow Y \text{ and } X \rightarrow W \rightarrow Z \rightarrow Y$$

2. Paths from X to Z are:

$$X \rightarrow W \rightarrow Z \text{ and } X \rightarrow W \rightarrow Y \rightarrow Z$$

Hence, every possible path from X must pass through the node W to travel to any node in $\{Y, Z\}$. The set $C = \{W\}$ separates $A = \{X\}$ from $B = \{Y, Z\}$.

$\therefore$ The distribution must satisfy the conditional independence property:

$$X \perp \{Y, Z\} \mid W$$

Solution 4

Solution 4 (a)

Given, the joint distribution P over (x_1, x_2, x_3) is given by the following table:

x_1	x_2	x_3	Pr
0	0	0	1/3
0	0	1	1/3
1	0	0	1/6
1	1	1	1/6

To check if variable Y is a deterministic function of X and Z , we check if, for every pair of values (x, z) that has a non-zero probability of occurring, there is only one possible value for y .

1. Test $x_1 = f(x_2, x_3)$

- Row 1: $(x_2 = 0, x_3 = 0)$ maps to $x_1 = 0$.
- Row 3: $(x_2 = 0, x_3 = 0)$ maps to $x_1 = 1$.

Hence, this single input maps to two different outputs.

2. Test $x_3 = f(x_1, x_2)$

- Row 1: $(x_1 = 0, x_2 = 0)$ maps to $x_3 = 0$.
- Row 2: $(x_1 = 0, x_2 = 0)$ maps to $x_3 = 1$.

Hence, this single input maps to two different outputs.

3. $x_2 = f(x_1, x_3)$

- Row 1: Input $(x_1 = 0, x_3 = 0)$ maps to $x_2 = 0$.
- Row 2: Input $(x_1 = 0, x_3 = 1)$ maps to $x_2 = 0$.
- Row 3: Input $(x_1 = 1, x_3 = 0)$ maps to $x_2 = 0$.
- Row 4: Input $(x_1 = 1, x_3 = 1)$ maps to $x_2 = 1$.

In every case, a given input pair (x_1, x_3) maps to a single, unique output for x_2 .

$\therefore$ The deterministic relationship is:

$$x_2 = x_1 \wedge x_3 \quad (\text{i.e., } x_2 \text{ is } 1 \iff x_1 = 1 \text{ and } x_3 = 1)$$

Solution 4

Solution 4 (b)

From *part (a)*, x_2 is a deterministic function of x_1 and x_3 , defined as:

$$x_2 = x_1 \wedge x_3$$

This means $P(x_2 \mid x_1, x_3)$ is deterministic. This strongly suggests a structure where x_1 and x_3 are parents of x_2 .

This implies the factorization:

$$P(x_1, x_2, x_3) = P(x_1)P(x_3)P(x_2 \mid x_1, x_3)$$

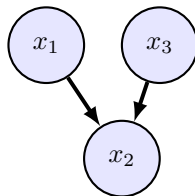
To test this independence, we first compute the marginals from the table:

- Marginal $P(x_1)$:
 - $\Pr(x_1 = 0) = \Pr(0, 0, 0) + \Pr(0, 0, 1) = 1/3 + 1/3 = 2/3$
 - $\Pr(x_1 = 1) = \Pr(1, 0, 0) + \Pr(1, 1, 1) = 1/6 + 1/6 = 1/3$
- Marginal $P(x_3)$:
 - $\Pr(x_3 = 0) = \Pr(0, 0, 0) + \Pr(1, 0, 0) = 1/3 + 1/6 = 3/6 = 1/2$
 - $\Pr(x_3 = 1) = \Pr(0, 0, 1) + \Pr(1, 1, 1) = 1/3 + 1/6 = 3/6 = 1/2$
- Joint $P(x_1, x_3)$: (Sum over x_2)
 - $\Pr(x_1 = 0, x_3 = 0) = \Pr(0, 0, 0) = 1/3$
 - $\Pr(x_1 = 0, x_3 = 1) = \Pr(0, 0, 1) = 1/3$
 - $\Pr(x_1 = 1, x_3 = 0) = \Pr(1, 0, 0) = 1/6$
 - $\Pr(x_1 = 1, x_3 = 1) = \Pr(1, 1, 1) = 1/6$

Now, check for independence: $\Pr(x_1, x_3) = \Pr(x_1)\Pr(x_3)$

- $\Pr(x_1 = 0)\Pr(x_3 = 0) = (2/3)(1/2) = 1/3$. This equals $\Pr(x_1 = 0, x_3 = 0)$.
- $\Pr(x_1 = 0)\Pr(x_3 = 1) = (2/3)(1/2) = 1/3$. This equals $\Pr(x_1 = 0, x_3 = 1)$.
- $\Pr(x_1 = 1)\Pr(x_3 = 0) = (1/3)(1/2) = 1/6$. This equals $\Pr(x_1 = 1, x_3 = 0)$.
- $\Pr(x_1 = 1)\Pr(x_3 = 1) = (1/3)(1/2) = 1/6$. This equals $\Pr(x_1 = 1, x_3 = 1)$.

It follows that, $P(x_1, x_3) = P(x_1)P(x_3)$ holds for all combinations, x_1 and x_3 are marginally independent. Hence, the minimal Bayes net is therefore the one that encodes $x_1 \perp x_3$ and the dependency $x_2 = f(x_1, x_3)$. This is the v-structure $x_1 \rightarrow x_2 \leftarrow x_3$.



$\therefore$ The minimal Bayes net is the v-structure $x_1 \rightarrow x_2 \leftarrow x_3$, as x_1 and x_3 are marginally independent, and x_2 is a deterministic function of both.

Solution 4

Solution 4 (c)

From *part(b)*, we know the minimal Bayes net is the v-structure $x_1 \rightarrow x_2 \leftarrow x_3$. The joint distribution thus factors as:

$$P(x_1, x_2, x_3) = P(x_1)P(x_3)P(x_2 \mid x_1, x_3)$$

This factorization yields three (Conditional) Probability Tables (CPTs).

1. CPT for x_1 : Find the marginal probability $P(x_1)$ by summing over x_2 and x_3 in the joint table.

$$\Pr(x_1 = 0) = \Pr(0, 0, 0) + \Pr(0, 0, 1) = \frac{1}{3} + \frac{1}{3} = \frac{2}{3}$$

$$\Pr(x_1 = 1) = \Pr(1, 0, 0) + \Pr(1, 1, 1) = \frac{1}{6} + \frac{1}{6} = \frac{1}{3}$$

x_1	$\Pr(x_1)$
0	$2/3$
1	$1/3$

2. CPT for x_3 : Find the marginal probability $P(x_3)$ by summing over x_1 and x_2 .

$$\Pr(x_3 = 0) = \Pr(0, 0, 0) + \Pr(1, 0, 0) = \frac{1}{3} + \frac{1}{6} = \frac{1}{2}$$

$$\Pr(x_3 = 1) = \Pr(0, 0, 1) + \Pr(1, 1, 1) = \frac{1}{3} + \frac{1}{6} = \frac{1}{2}$$

x_3	$\Pr(x_3)$
0	$1/2$
1	$1/2$

3. CPT for x_2 : From *part (a)*, $x_2 = x_1 \wedge x_3$. This conditional probability is deterministic.

x_1	x_3	$\Pr(x_2 = 0 \mid x_1, x_3)$	$\Pr(x_2 = 1 \mid x_1, x_3)$
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

$\therefore$ The three tables are $P(x_1)$ (with $\Pr(x_1 = 1) = 1/3$), $P(x_3)$ (with $\Pr(x_3 = 1) = 1/2$), and the deterministic CPT for $P(x_2 \mid x_1, x_3)$ which implements the $x_2 = x_1 \wedge x_3$ function.

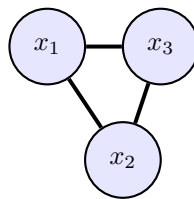
Solution 4

Solution 4 (d)

To find the minimal undirected graph (UGM), we "moralize" the minimal Bayes net from *part (b)*.

1. Start with the Bayes Net: From *part (b)*, the graph was the v-structure $x_1 \rightarrow x_2 \leftarrow x_3$.
2. Add moral edge: The nodes x_1 and x_3 are both parents of x_2 . We must add an undirected edge between them.
3. Drop Arrowheads: Convert all directed edges ($x_1 \rightarrow x_2$ and $x_3 \rightarrow x_2$) into undirected edges.

This process results in a fully connected graph (a 3-clique) with edges (x_1, x_2) , (x_2, x_3) , and the new moral edge (x_1, x_3) .



$\therefore$ The minimal undirected graph can be seen above.

Solution 5

Solution 5 (a)

The joint distribution from the notebook is:

$$\Pr(X, Y) \propto \exp \left(\sum_p X_p Y_p + \sum_{p \sim q} Y_p Y_q \right)$$

For Gibbs sampling, the full conditional probability $\Pr(Y_p \mid Y_{\setminus p}, X)$ is proportional to all terms in the joint distribution that involve Y_p :

$$\Pr(Y_p \mid Y_{\setminus p}, X) \propto \exp \left(X_p Y_p + \sum_{q \in \mathcal{N}(p)} Y_p Y_q \right)$$

where $\mathcal{N}(p)$ is the set of neighbors of pixel p . We can factor out Y_p :

$$\Pr(Y_p \mid Y_{\setminus p}, X) \propto \exp \left(Y_p \cdot \left(X_p + \sum_{q \in \mathcal{N}(p)} Y_q \right) \right)$$

The local evidence is defined as:

$$E_p = X_p + \sum_{q \in \mathcal{N}(p)} Y_q$$

Substituting E_p the equation simplifies to:

$$\Pr(Y_p \mid Y_{\setminus p}, X) \propto \exp(Y_p E_p)$$

Now, normalize over the two possible states $Y_p \in \{-1, +1\}$:

$$\Pr(Y_p = +1 \mid \dots) = \frac{\exp((+1) \cdot E_p)}{\exp((+1) \cdot E_p) + \exp((-1) \cdot E_p)} = \frac{e^{E_p}}{e^{E_p} + e^{-E_p}}$$

Finally, multiply the numerator and denominator by e^{-E_p} (1) and we arrive at the following:

$$\Pr(Y_p = +1 \mid \dots) = \frac{1}{1 + e^{-2E_p}}$$

$\therefore$ The precise expression is:

$$\Pr(Y_p = +1 \mid Y_{\setminus p}, X) = \frac{1}{1 + \exp(-2E_p)} \quad , \text{where } E_p = X_p + \sum_{q \in \mathcal{N}(p)} Y_q$$

Solution 5

Solution 5 (b)

Python Code: Gibbs sampler

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import cv2
4  import math
5
6  def load_image(filename):
7      img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
8      img = img/255.0
9      img[img>=0.5]=1.0
10     img[img<0.5]=0.0
11     img = img*2-1
12     return img
13
14     # Load in both the degraded image and the original
15     img = load_image("noisy_bear.png")
16     gt = load_image("original_bear.png")
17     M,N = img.shape[0], img.shape[1]
18
19     # Create a version of img that is padded with zeros all around
20     padded_image = np.zeros((img.shape[0]+2, img.shape[1]+2)) #Padding image to make some
        computations easier
21     padded_image[1:-1,1:-1] = img
22
23     def gibbs_step(Y, X):
24         Y_new = np.copy(Y)
25         M, N = Y.shape
26
27         # loop over every pixel
28         for i in range(M):
29             for j in range(N):
30                 # sum over neighbors
31                 neighbors_sum = 0
32                 if i > 0: neighbors_sum += Y_new[i-1, j]
33                 if i < M-1: neighbors_sum += Y_new[i+1, j]
34                 if j > 0: neighbors_sum += Y_new[i, j-1]
35                 if j < N-1: neighbors_sum += Y_new[i, j+1]
36
37                 # E_p = eta * X_p + beta * neighbors_sum (eta=1 and beta=1)
38                 E_p = X[i, j] + neighbors_sum
39
40                 # prob of Y_p = +1 -> prob_plus_one = 1.0 / (1.0 + np.exp(-2.0 * E_p))
41                 if E_p * 2 > 700: # Avoid overflow in exp
42                     prob_plus_one = 1.0
43                 elif E_p * 2 < -700:
44                     prob_plus_one = 0.0
45                 else:
46                     prob_plus_one = 1.0 / (1.0 + np.exp(-2.0 * E_p))
47
48                 # sample from Bernoulli(prob_plus_one)
49                 if np.random.rand() < prob_plus_one:
50                     Y_new[i, j] = 1
51                 else:
52                     Y_new[i, j] = -1
53
54             return Y_new
55
56     # initialize Y and set X to the img
57     X = img
58     Y = np.random.choice([-1, 1], size=X.shape)
59     num_iterations = 50
60     images_to_show = []
61     iterations_to_save = [0, 1, 5, 10, 49]
62

```

```
63 for t in range(num_iterations):
64     if t in iterations_to_save:
65         images_to_show.append(np.copy(Y))
66     Y = gibbs_step(Y, X)
67
68 # plot
69 fig, axes = plt.subplots(1, len(images_to_show), figsize=(20, 4))
70 for ax, img, t in zip(axes, images_to_show, iterations_to_save):
71     ax.imshow(img, cmap='gray')
72     ax.set_title(f'Iteration {t}')
73     ax.axis('off')
74 plt.show()
```

Images

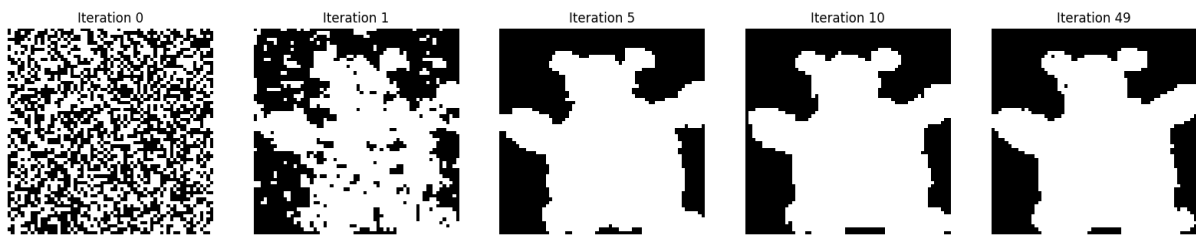


Figure 2: Image 0,1,5,10,49 while Markov chain is missing

Solution 5

Solution 5 (c)

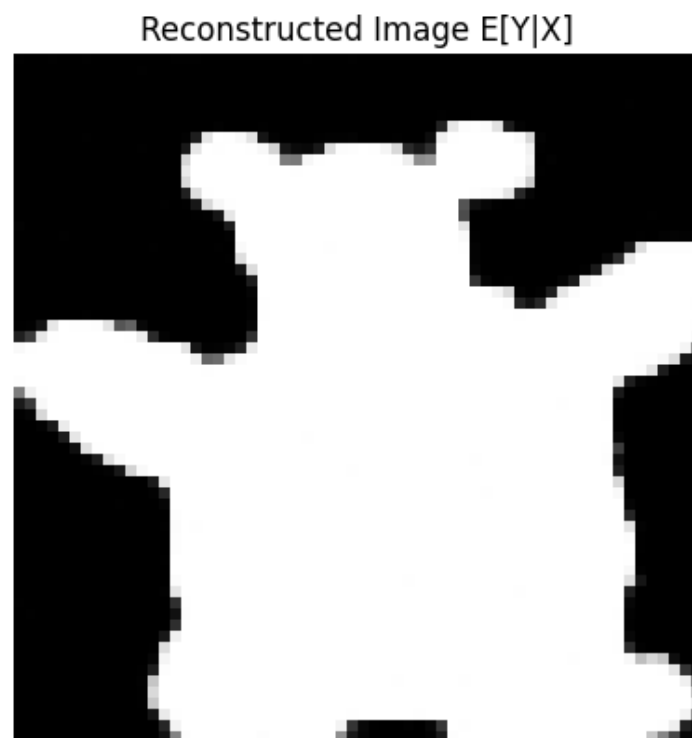
Python Code: Reconstruct Image

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 def get_posterior_mean(Y, X, burn_in, num_samples):
4     M, N = X.shape
5     Y_samples_sum = np.zeros((M, N), dtype=float)
6
7     # run the chain
8     for t in range(burn_in + num_samples):
9         Y = gibbs_step(Y, X)
10        # After burn-in, start collecting samples
11        if t >= burn_in:
12            Y_samples_sum += Y
13        # calc average
14        posterior_mean = Y_samples_sum / num_samples
15    return posterior_mean
16
17 X= img
18 Y_init = np.random.choice([-1, 1], size=X.shape)
19 posterior_mean_image = get_posterior_mean(Y_init, X, burn_in=50, num_samples=200)
20
21 # plot
22 plt.imshow(posterior_mean_image, cmap='gray', vmin=-1, vmax=1)
23 plt.title('Reconstructed Image E[Y|X]')
24 plt.axis('off')
25 plt.show()

```

Reconstructed Image



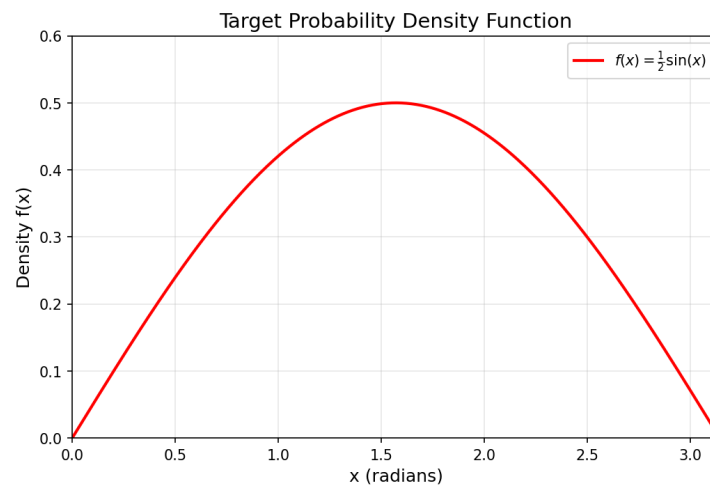
Solution 6

Solution 6 (a)

Python Code: Plot $f(x)$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # set the function and domain
5 f = lambda x: 0.5 * np.sin(x)
6 x_domain = np.linspace(0, np.pi, 400)
7 y_values = f(x_domain)
8
9 # plot
10 fig, ax = plt.subplots(figsize=(8, 5))
11 ax.plot(x_domain, y_values, 'r-', linewidth=2,
12         label=r'$f(x) = \frac{1}{2}\sin(x)$')
13 ax.set_xlabel('x (radians)', fontsize=12)
14 ax.set_ylabel('Density f(x)', fontsize=12)
15 ax.set_title('Target Probability Density Function', fontsize=14)
16 ax.legend(fontsize=10)
17 ax.grid(True, alpha=0.3)
18 ax.set_xlim(0, np.pi)
19 ax.set_ylim(0, 0.6) # Max is 0.5, give a little space
20 plt.savefig('q6a_plot.png', dpi=150, bbox_inches='tight')
21 plt.show()
```

Plot



Solution 6

Solution 6 (b)

1. Suitable Proposal $g(x)$ The target density $f(x)$ is defined on the interval $[0, \pi]$. The simplest proposal distribution $g(x)$ that covers this support is the Uniform distribution on $[0, \pi]$.

- Its support, $[0, \pi]$, matches the support of $f(x)$.
- $f(x) > 0$ only when $x \in (0, \pi)$, and $g(x) > 0$ on this entire interval.

The probability density function (PDF) for $g(x) = \text{Uniform}(0, \pi)$ is:

$$g(x) = \frac{1}{\pi - 0} = \frac{1}{\pi} \quad \text{for } x \in [0, \pi]$$

2. Suitable Upper Bound M We need to find the smallest constant M such that $f(x) \leq Mg(x)$, which is equivalent to finding the maximum of the ratio $f(x)/g(x)$:

$$M = \sup_{x \in [0, \pi]} \frac{f(x)}{g(x)}$$

We compute this ratio:

$$\frac{f(x)}{g(x)} = \frac{\frac{1}{2} \sin x}{\frac{1}{\pi}} = \frac{\pi}{2} \sin x$$

To find the maximum, we find the maximum of $\sin x$ on the interval $[0, \pi]$, which is 1 (at $x = \pi/2$).

Hence, the tightest possible upper bound M is:

$$M = \frac{\pi}{2} \cdot \left(\max_{x \in [0, \pi]} \sin x \right) = \frac{\pi}{2} \cdot 1 = \frac{\pi}{2}$$

$\therefore$ A suitable proposal is $g(x) = \text{Uniform}(0, \pi)$ with PDF $g(x) = 1/\pi$. The corresponding tightest upper bound is $M = \frac{\pi}{2}$.

Solution 6

Solution 6 (c)

Python Code: Rejection Sampler

```
1 import numpy as np
2
3 # set seed
4 np.random.seed(42)
5
6 N_ACCEPT = 10000
7 accepted_samples = []
8 total_draws = 0
9
10 # Parameters from part (b)
11 f = lambda x: 0.5 * np.sin(x)
12 g = lambda x: 1.0 / np.pi
13 M = np.pi / 2.0
14
15 # loop until we have 10,000 accepted samples
16 while len(accepted_samples) < N_ACCEPT:
17     total_draws += 1
18
19     # 1. Sample x from g(x) = Uniform(0, pi)
20     x_proposal = np.random.uniform(0, np.pi)
21
22     # 2. Sample u from Uniform(0, 1)
23     u = np.random.rand()
24
25     # 3. Calculate acceptance probability
26     # p_accept = f(x) / (M * g(x)) = sin(x)
27     acceptance_prob = np.sin(x_proposal)
28
29     # 4. Accept/Reject
30     if u <= acceptance_prob:
31         accepted_samples.append(x_proposal)
32
33 # Print the results
34 print(f"Target number of samples: {N_ACCEPT}")
35 print(f"Total samples drawn from g: {total_draws}")
36 print(f"Empirical acceptance rate: {N_ACCEPT / total_draws:.4f}")
37 print(f"Theoretical acceptance rate (2/pi): {2.0 / np.pi:.4f}")
```

∴ see output below

```
Target number of samples: 10000
Total samples drawn from g: 15529
Empirical acceptance rate: 0.6440
Theoretical acceptance rate (2/pi): 0.6366
```

Solution 6

Solution 6 (d)

We use the `accepted.samples` list (containing 10,000 samples) that was generated in part 6(c). We use Matplotlib's `hist` function, specifying `bins=50` and, most importantly, `density=True`. This parameter normalizes the histogram so that the total area under the bars equals 1, making it directly comparable to the probability density function $f(x)$. We then overlay the plot of $f(x)$ from 6(a) to visually confirm the match.

Python Code: Plot Histogram

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # used samples from part (c)
5
6 # 1. Define the true function and domain (from 6a)
7 f = lambda x: 0.5 * np.sin(x)
8 x_domain = np.linspace(0, np.pi, 400)
9 y_true = f(x_domain)
10
11 # plot
12 fig, ax = plt.subplots(figsize=(8, 5))
13 ax.hist(accepted_samples, bins=50, density=True,
14         label='Sampled Histogram (10k samples)',
15         alpha=0.7, edgecolor='k', linewidth=0.5)
16 ax.plot(x_domain, y_true, 'r-', linewidth=3,
17         label=r'True Density $f(x) = \frac{1}{2}\sin(x)$')
18 ax.set_xlabel('x (radians)', fontsize=12)
19 ax.set_ylabel('Density', fontsize=12)
20 ax.set_title('Rejection Sampling: Histogram vs. True Density', fontsize=14)
21 ax.legend(fontsize=10)
22 ax.grid(True, alpha=0.3)
23 ax.set_xlim(0, np.pi)
24 plt.savefig('q6d_histogram.png', dpi=150, bbox_inches='tight')
25 plt.show()
```

Listing 1: Python code to plot the histogram and true density.

Resulting Plot

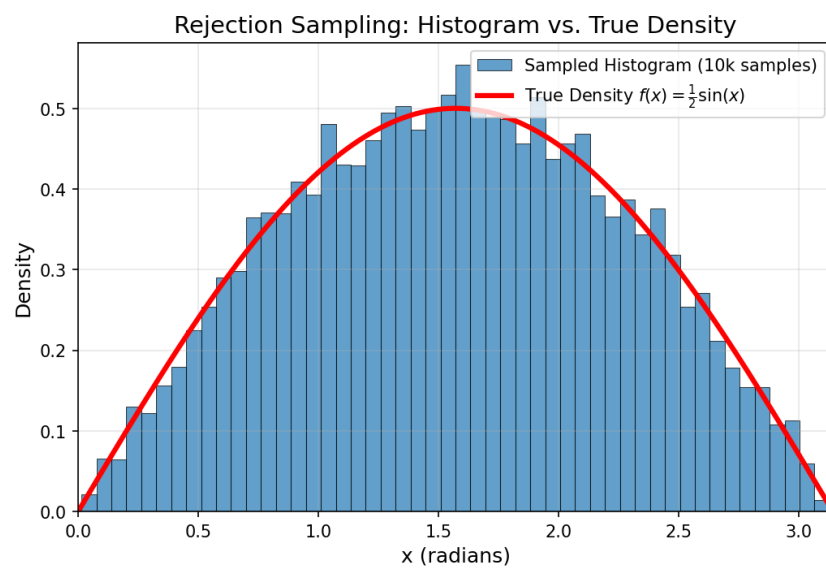


Figure 3: Normalized histogram of 10,000 accepted samples overlaid with the true target density $f(x)$.